Pipeline Core Language

执行类别与并发操作语义

1　静态对象与执行类别

令 Pipe、Addr、Lock 分别表示 pipeline、存储单元地址和锁的标识集合。一个程序使用有限集合 $\mathcal{P} \subseteq \mathsf{Pipe}$。pipeline 标识包含所属设备、core 和执行队列的身份。每个 pipeline 具有独立的顺序命令、局部变量和当前操作状态。不同 pipeline 的步骤可以交错。

每个地址标识一个原子访问单元，有限地址集合表示访问区域。共享内存 $\mu : \mathsf{Addr} \rightharpoonup \mathsf{Val}$ 保存地址和值。源程序中具有别名的地址在翻译后必须保持相等。

执行类别由静态函数指定：

$$\mathsf{ExecKind} = \{\mathsf{Cube}, \mathsf{Vector}, \mathsf{Scalar}, \mathsf{MTE}\}, \qquad \mathrm{kind} : \mathsf{Pipe} \to \mathsf{ExecKind}.$$

例如 $\mathrm{kind}(p_c) = \mathsf{Cube}$、$\mathrm{kind}(p_v) = \mathsf{Vector}$。访问权限和函数支持关系由静态环境 Γ 给出：

$$\mathrm{Supports}(p, f, \bar{\tau}) \in \mathsf{Bool}, \quad \mathrm{CanRead}(p, a) \in \mathsf{Bool},$$
$$\mathrm{CanWrite}(p, a) \in \mathsf{Bool}, \quad \mathrm{TypeOK}(a, v) \in \mathsf{Bool}.$$

函数签名中的类型可以携带向量和矩阵形状。计算函数 f 是定义在指定输入类型上的确定性纯偏函数；其数值精度和舍入规则由具体实例给出。计算函数不访问共享内存，也不操作事件和锁。

事件标识为 $\varepsilon = (s, p, q, k)$，分别表示同步域、源 pipeline、目标 pipeline 和事件编号。Γ 声明事件及每个同步域的参与者；事件的两个端点必须属于相应同步域。每把锁 ℓ 声明允许参与的 pipeline 集合 $\mathrm{Participants}(\ell) \subseteq \mathcal{P}$。

2　语法

令 x 为局部变量，v 为值，g 为纯表达式函数，k 为执行类别。值包括标量、布尔值、地址、有限地址集合、地址到值的有限映射、向量、矩阵和 unit。语法为：

$$\begin{aligned}
e &::= v \mid x \mid g(e_1, \ldots, e_n),\\
o &::= \mathsf{read}(e_R) \mid \mathsf{write}(e_R, e_V) \mid \mathsf{computation}_{k,f}(e_1, \ldots, e_n),\\
a &::= x := e \mid x := o \mid \mathsf{setflag}(\varepsilon) \mid \mathsf{waitflag}(\varepsilon) \mid \mathsf{lock}(\ell) \mid \mathsf{unlock}(\ell),\\
C &::= \mathsf{skip} \mid a \mid C; C \mid \mathsf{if}\ e\ \mathsf{then}\ C\ \mathsf{else}\ C \mid \mathsf{while}\ e\ \mathsf{do}\ C,\\
P &::= \mathsf{parallel}\{p_1 : C_1, \ldots, p_n : C_n\}.
\end{aligned}$$

程序中各个 pipeline 标识互不相同，且 $\mathcal{P} = \{p_1, \ldots, p_n\}$。$[\![e]\!]_\rho$ 在局部环境 ρ 中求值。e_R 求值得到有限地址集合 R；e_V 求值得到定义域为 R 的有限映射 V。读取返回地址值映射，写入返回 `unit`。例如 `unit:=write(R,V)` 中的 `unit` 是接收返回值的局部变量。

同一 pipeline 的命令按程序顺序执行：操作完成、结果写入局部变量之后，该 pipeline 才能执行后续命令。一个操作可以经历多个语义步骤，这些步骤可以与其他 pipeline 的步骤交错。

3 状态与操作初始化

运行状态和 pipeline 局部状态分别为

$$\Sigma = \langle \mu, F, L, T \rangle, \qquad T(p) = \langle C_p, \rho_p, d_p \rangle.$$

F 将已声明事件映射到二值标志，L 将已声明锁映射到空闲标记 $\perp$ 或持有者 pipeline。C_p 是剩余命令，ρ_p 是局部环境，d_p 是空闲标记 $\perp$ 或当前操作的进度：

$$d ::= \mathrm{Reading}(U, B) \mid \mathrm{Writing}(U, V) \mid \mathrm{Computing}(k, f, \bar{v}) \mid \mathrm{Done}(v).$$

U 表示尚未访问的地址集合；B 保存已经读取的值；V 保存写入值。执行计算期间保存执行类别、函数和已求值的参数。$\mathrm{Done}(v)$ 表示内部执行已经完成，结果尚待写入局部环境。

在当前局部环境中初始化操作：

$$\begin{aligned}
\mathrm{init}_\Gamma(\rho, \mathsf{read}(e_R)) &= \mathrm{Reading}(R, \varnothing) && \text{当} [\![e_R]\!]_\rho = R, \\
\mathrm{init}_\Gamma(\rho, \mathsf{write}(e_R, e_V)) &= \mathrm{Writing}(R, V) && \text{当} [\![e_R]\!]_\rho = R,\ [\![e_V]\!]_\rho = V, \\
\mathrm{init}_\Gamma(\rho, \mathsf{computation}_{k,f}(\bar{e})) &= \mathrm{Computing}(k, f, \bar{v}) && \text{当} [\![\bar{e}]\!]_\rho = \bar{v}.
\end{aligned}$$

初始化后的状态还必须满足 $\mathrm{Valid}_\Gamma(p, \mu, d)$：

$$\begin{aligned}
\mathrm{Valid}(p, \mu, \mathrm{Reading}(R, \varnothing)) &\iff R \subseteq \mathrm{dom}(\mu) \wedge \forall a \in R.\ \mathrm{CanRead}(p, a), \\
\mathrm{Valid}(p, \mu, \mathrm{Writing}(R, V)) &\iff R \subseteq \mathrm{dom}(\mu) \wedge \mathrm{dom}(V) = R \wedge \\
&\qquad \forall a \in R.\ \mathrm{CanWrite}(p, a) \wedge \mathrm{TypeOK}(a, V(a)), \\
\mathrm{Valid}(p, \mu, \mathrm{Computing}(k, f, \bar{v})) &\iff \mathrm{kind}(p) = k \wedge \mathrm{Supports}(p, f, \mathrm{types}(\bar{v})) \wedge f(\bar{v}) \text{ 有定义}.
\end{aligned}$$

表达式求值失败、区域或数值类型错误、权限不足、函数未定义及计算类别不匹配均产生错误。

初始状态中，$T(p) = (C_p, \rho_p, \perp)$，每个事件标志为 0，每把锁空闲。初始内存符合 TypeOK，初始局部环境只包含已声明的 pipeline；Γ 的事件、锁和计算函数声明满足上述条件。

4 带标签的小步操作语义

下列规则使用执行上下文 $E ::= [\,] \mid E; C$。写作 $T[p \mapsto (C, \rho, d)]$ 的更新只改变 p 的局部状态；未列出的全局分量保持原值。标签中的 start 和 complete 记录一个操作的内部执行步骤，不属于程序语法。

4.1 顺序控制与操作

局部赋值、顺序组合、条件和循环按以下规则执行：

$$\frac{T(p) = (E[x := e], \rho, \bot) \quad [\![e]\!]_\rho = v}{\Sigma \xrightarrow{\text{assign}(p,x,v)} \Sigma[T(p) \mapsto (E[\mathsf{skip}], \rho[x \mapsto v], \bot)]}$$

$$\frac{T(p) = (E[\mathsf{skip}; C], \rho, \bot)}{\Sigma \xrightarrow{\text{control}(p)} \Sigma[T(p) \mapsto (E[C], \rho, \bot)]}$$

$$\frac{T(p) = (E[\mathsf{if}\ e\ \mathsf{then}\ C_1\ \mathsf{else}\ C_2], \rho, \bot) \quad [\![e]\!]_\rho = b}{\Sigma \xrightarrow{\text{control}(p)} \Sigma[T(p) \mapsto (E[C_b], \rho, \bot)]}$$

其中 $C_{\mathsf{true}} = C_1$，$C_{\mathsf{false}} = C_2$。循环展开为 $\mathsf{if}\ e\ \mathsf{then}\ (C; \mathsf{while}\ e\ \mathsf{do}\ C)\ \mathsf{else}\ \mathsf{skip}$，并产生一个 control($p$) 标签。

操作首先求值参数并检查合法性：

$$\frac{T(p) = (E[x := o], \rho, \bot) \quad \text{init}_\Gamma(\rho, o) = d \quad \text{Valid}_\Gamma(p, \mu, d)}{\Sigma \xrightarrow{\text{start}(p,o)} \Sigma[T(p) \mapsto (E[x := o], \rho, d)]}.$$

当前命令保持为 $x := o$，直到操作的结果被写入局部环境。读取和写入每次访问一个尚未处理的地址：

$$\frac{T(p) = (E[x := o], \rho, \text{Reading}(U, B)) \quad a \in U \quad \mu(a) = v}{\Sigma \xrightarrow{\text{read}(p,a,v)} \Sigma[T(p) \mapsto (E[x := o], \rho, \text{Reading}(U \setminus \{a\}, B[a \mapsto v]))]}$$

$$\frac{T(p) = (E[x := o], \rho, \text{Writing}(U, V)) \quad a \in U \quad V(a) = v}{\Sigma \xrightarrow{\text{write}(p,a,v)} \Sigma[\mu \mapsto \mu[a \mapsto v]][T(p) \mapsto (E[x := o], \rho, \text{Writing}(U \setminus \{a\}, V))]}.$$

区域内部地址的处理顺序由调度非确定地选择。地址集合为空时，读取状态转为 Done(B)，写入状态转为 Done(unit)；计算状态 Computing($k, f, \bar{v}$) 转为 Done($f(\bar{v})$)。这些步骤产生 control(p) 标签并保持共享内存。随后使用规则

$$\frac{T(p) = (E[x := o], \rho, \text{Done}(v))}{\Sigma \xrightarrow{\text{complete}(p,x,v)} \Sigma[T(p) \mapsto (E[\mathsf{skip}], \rho[x \mapsto v], \bot)]}.$$

因此，同一 pipeline 在当前操作完成之前不能执行下一条命令；其他 pipeline 可以继续执行。

4.2 事件和锁

对已声明事件 $\varepsilon = (s, p, q, k)$，源 pipeline 设置事件，目标 pipeline 消费事件：

$$\frac{T(p) = (E[\mathsf{setflag}(\varepsilon)], \rho, \bot)}{\Sigma \xrightarrow{\text{set}(p,\varepsilon)} \Sigma[F \mapsto F[\varepsilon \mapsto 1]][T(p) \mapsto (E[\mathsf{skip}], \rho, \bot)]}$$

$$\frac{T(q) = (E[\mathsf{waitflag}(\varepsilon)], \rho, \bot) \quad F(\varepsilon) = 1}{\Sigma \xrightarrow{\text{wait}(q,\varepsilon)} \Sigma[F \mapsto F[\varepsilon \mapsto 0]][T(q) \mapsto (E[\mathsf{skip}], \rho, \bot)]}.$$

重复设置保持标志为 1；等待成功时将标志置为 0。标志为 0 时等待没有可用转移。根据顺序语义，执行设置事件时，源 pipeline 的前序操作已经完成。

令 $p \in \mathrm{Participants}(\ell)$。获取和释放规则为

$$\frac{T(p) = (E[\mathsf{lock}(\ell)], \rho, \bot) \quad L(\ell) = \bot}{\Sigma \xrightarrow{\mathrm{acquire}(p,\ell)} \Sigma[L \mapsto L[\ell \mapsto p]][T(p) \mapsto (E[\mathsf{skip}], \rho, \bot)]}$$

$$\frac{T(p) = (E[\mathsf{unlock}(\ell)], \rho, \bot) \quad L(\ell) = p}{\Sigma \xrightarrow{\mathrm{release}(p,\ell)} \Sigma[L \mapsto L[\ell \mapsto \bot]][T(p) \mapsto (E[\mathsf{skip}], \rho, \bot)]}.$$

锁被持有时，其他参与者的获取步骤不可用。事件未声明、事件端点不匹配、锁未声明、参与权限不足或者释放者不持有锁时进入错误状态。

4.3 全局调度、错误与终止

全局步骤非确定地选择一个 $p \in \mathcal{P}$，执行上述一个可用步骤。局部控制、操作进度、内存访问及同步操作都属于该 pipeline。非法命令产生 $\mathrm{error}(p)$ 标签并进入无后继的 Error。等待标志和等待锁时条件暂未满足属于阻塞。

正常终止定义为所有 pipeline 均满足 $C_p = \mathsf{skip}$ 且 $d_p = \bot$。未正常终止且没有任何可用步骤的状态是全局死锁。部分 pipeline 长期等待而其他 pipeline 持续执行的情形由活性性质描述。

程序语义包含所有合法初态出发的最大有限轨迹和无限轨迹。可选的弱公平条件要求持续可执行的 pipeline 最终获得调度；基本语义不强制公平。

5 事件顺序与数据竞争

对有限标签轨迹 τ，记 τ_i 为第 i 个事件，owner 返回事件所属 pipeline。基本顺序边包括：

1. 程序顺序：$i < j$ 且 $\mathrm{owner}(\tau_i) = \mathrm{owner}(\tau_j)$。这包括该 pipeline 的读写步骤和同步步骤。
2. 事件同步：$\tau_i = \mathrm{set}(p, \varepsilon)$、$\tau_j = \mathrm{wait}(q, \varepsilon)$，$i < j$，中间没有对 ε 的设置或等待。
3. 锁同步：$\tau_i = \mathrm{release}(p, \ell)$、$\tau_j = \mathrm{acquire}(q, \ell)$，$i < j$，中间没有对 ℓ 的获取。

happens-before 是这些边的传递闭包，记为 hb。两个不同位置 i, j 构成冲突，当且仅当它们访问同一地址且至少一个是写入。数据竞争为

$$\mathrm{Race}_\tau(i, j) \iff i \neq j \wedge \mathrm{Conflict}_\tau(i, j) \wedge \neg(i \ hb \ j) \wedge \neg(j \ hb \ i).$$

同一 pipeline 的任意两个先后访问都由程序顺序连接。区域访问可以与其他 pipeline 的访问交错，因而可以产生竞争和混合读取结果。

6 可选的顺序锁策略

普通锁语义不指定竞争者的获取次序。可选扩展使用 $\mathrm{order} : \mathsf{Lock} \times \mathbb{N} \to \mathsf{Pipe}$，以及每把锁的轮次 $\mathrm{turn}(\ell)$。初始轮次为 0；获取时除了普通规则的前提，还要求 $\mathrm{order}(\ell, \mathrm{turn}(\ell)) = p$；成功释放后将轮次加一。每个轮次的选定 pipeline 必须属于该锁的参与者集合。忽略轮次状态后，扩

展的每个步骤均为普通语义的步骤。无限轮转顺序 $\mathrm{order}(\ell, n) = n \bmod 2$ 可用于参与者为 $0, 1$ 的锁。

7 形式化范围与参考

Rocq 定义位于 `rocq/CoreLanguage/`，包含语法、状态、带标签小步关系、执行轨迹、happens-before、数据竞争、顺序锁和具体执行推导。Rocq 证明覆盖操作启动所需的空闲条件、完成时的结果状态、计算类别检查、正常终止状态没有后继、happens-before 的顺序性质和顺序锁性质。实际硬件的缓存传播、队列容量、执行时延以及硬件程序翻译到本语言的 refinement 需要针对具体目标建立。将安全结论传递到硬件时还需证明访存、别名、同步作用域、执行类别和事件顺序得到保持。

事件的二值消费含义参考 Ascend 文档 [1, 2]；HIVM 同步操作的定义可见仓库中的 `vendor/ascendnpu-ir/upstream/bishengir/include/bishengir/Dialect/HIVM/IR/HIVMSynchronizationOps.td`。锁的 acquire/release 含义参考 Triton 原子操作接口 [3, 4]，具体翻译由目标平台给出。

参考文献

[1] Ascend C，*SetFlag/WaitFlag*。https://asc.gitcode.com/api/SIMD-API/basic_api/sync_control/intra_core_sync/SetFlag-WaitFlag.html。

[2] Ascend C，*Basic Architecture*，CANN 9.0。https://www.hiascend.com/document/detail/en/CANNCommunityEdition/900/programug/Ascendcopdevg/atlas_ascendc_10_0008.html。

[3] Triton，*triton.language.atomic_cas*。https://triton-lang.org/main/python-api/generated/triton.language.atomic_cas.html。

[4] Triton，*triton.language.atomic_xchg*。https://triton-lang.org/main/python-api/generated/triton.language.atomic_xchg.html。

8 具体执行例子

8.1 同一 pipeline 的操作顺序

```
parallel {
  pv:
    x := computation[Vector, add](2, 3);
    y := computation[Vector, add](x, 10);
}
```

第一条计算执行期间，d_{p_v} 依次为 Computing(Vector, add, $[2,3]$) 和 Done(5)。结果写入 x 后，第一条命令变成 skip；消除顺序组合中的 skip 后，第二条计算才能求值 x 并得到 $y = 15$。

8.2 Cube 和 Vector pipeline 的并行计算

设 $\mathrm{kind}(p_c) = \mathsf{Cube}$、$\mathrm{kind}(p_v) = \mathsf{Vector}$，$A = \left(\begin{smallmatrix}1 & 2\\ 3 & 4\end{smallmatrix}\right)$，$B = \left(\begin{smallmatrix}5 & 6\\ 7 & 8\end{smallmatrix}\right)$。

```
parallel {
  pv: z := computation[Vector, vadd]([1, 2], [10, 20]);
  pc: c := computation[Cube, matmul](A, B);
}
```

一条合法执行依次启动 Vector 计算、启动 Cube 计算、完成 Vector 计算并写入 $z = [11, 22]$，再完成 Cube 计算并写入 $c = \left(\begin{smallmatrix}19 & 22\\ 43 & 50\end{smallmatrix}\right)$。两次计算分别保存在所属 pipeline 的当前操作状态中。将 Cube 计算放在 p_v 中会因执行类别不匹配进入错误状态。

8.3 区域读取与写入交错

初始内存满足 $\mu(0) = \mu(1) = 0$，两个 pipeline 均具有相应访问权限：

```
parallel {
  p0: result := read({0, 1});
  p1: unit := write({0, 1}, {0: 41, 1: 42});
}
```

一条合法执行先由 p_0 读取地址 0 的值 0，再由 p_1 写入地址 $0, 1$，最后由 p_0 读取地址 1 的值 42。返回的映射为 $\{0 \mapsto 0, 1 \mapsto 42\}$，最终内存为 $\{0 \mapsto 41, 1 \mapsto 42\}$。这些逐地址步骤对应 `MemoryExamples.v` 中经过证明的执行轨迹。

8.4 事件与临界区

设 ε 的源为 p_0、目标为 p_1，初始标志为 0：

```
parallel {
  p0: unit := write({a}, {a: 41}); setflag(epsilon);
  p1: waitflag(epsilon); result := read({a});
```

```
}
```

p_0 完成写入后才能设置事件；p_1 消费事件后读取地址 a。当没有其他写入时，读取结果为 41。

两个 pipeline 也可以使用同一把锁保护地址 a：

```
parallel {
  p0: lock(l); unit := write({a}, {a: 5}); unlock(l);
  p1: lock(l); result := read({a}); unlock(l);
}
```

每个 pipeline 的访存操作完成后才执行释放锁。锁的获取顺序决定两个临界区的先后顺序。

8.5 循环等待

设事件 ε_{01} 从 p_0 发往 p_1，ε_{10} 从 p_1 发往 p_0，初始标志均为 0：

```
parallel {
  p0: waitflag(e10); setflag(e01);
  p1: waitflag(e01); setflag(e10);
}
```

两个 pipeline 的当前等待都无法执行，程序未终止且不存在其他可用步骤，因此初始状态满足全局死锁定义。